



코드가 실제로 작동하는 상세 순서와 설명

아래는 `Car car1 = new Car("현대", 2025, "SUV")` 코드를 기준으로, 각 단계가 어떻게 동작하는지 자세히 설명한 것입니다.

1. 자식 클래스(Car) 객체 생성 명령 실행

```
Car car1 = new Car("현대", 2025, "SUV");
```

설명: Main에서 Car 타입 객체를 만들려고 호출함.

2. Car 생성자 진입

```
public Car(String brand, int year, String type)
```

설명: `brand="현대", year=2025, type="SUV"` 값이 들어옴. 여기서 바로 아래 명령이 실행됨:

3. Car 생성자에서 먼저 부모(Vehicle)의 생성자 호출

```
super(brand, year); // 부모 클래스 생성자 호출
```

설명: Car 객체가 만들어지기 전에 반드시 Vehicle(부모)의 생성자가 먼저 실행됨. 값이 "현대", 2025로 넘어감.^{[1] [2] [3]}

4. Vehicle 생성자에서 brand, year 값 초기화

```
Vehicle(String brand, int year) {  
    this.brand = brand;  
    this.year = year;  
}
```

설명: 여기서 부모의 private 변수 brand에 "현대"를, year에 2025를 각각 저장함.

5. Vehicle 생성자 종료 → Car 생성자 나머지 코드 실행

```
this.type = type;
```

설명: 부모 생성자 실행 끝나고, 자식(Car) 생성자에서 type 변수에 "SUV"를 저장함.

6. Car 객체 생성 완료 → showInfo() 실행

```
car1.showInfo();
```

설명: Car 객체가 완전히 생성된 뒤에 info 출력 메서드를 호출함.

7. 오버라이딩된 showInfo() 실행

```
@Override
public void showInfo() {
    super.showInfo();
    System.out.println("\n차종: " + type);
}
```

설명:

- 첫 줄에서 super.showInfo()를 호출, 즉 부모(Vehicle)의 showInfo() 실행
- 두 번째 줄에서 Car의 type을 출력

(1) 부모의 showInfo() 동작

```
public void showInfo() {
    System.out.printf("브랜드: %s, 연식: %d년", brand, year);
}
```

설명: 이 부분만큼은 부모 클래스 내부이므로, private 멤버의 접근에 아무런 제한이 없음 (brand와 year 모두 현재 car1 객체의 값을 보여줌).

(2) 자식의 showInfo() 동작

- 이어서 Car 클래스의 type(차종) 값을 추가로 화면에 출력

실제 출력 결과

```
브랜드: 현대, 연식: 2025년
차종: SUV
```

정리

- 자식 객체 생성시, 제일 먼저 부모의 생성자가 실행되고, 그 다음 자식의 생성자가 실행된다.
- 자식 생성자에서 부모 생성자를 명시적으로 super()로 호출해야 원하는 값이 초기화된다.
- 부모의 private 변수는 부모의 자신의 코드 안에서만 자유롭게 쓰일 수 있고, 자식은 부모 메서드를 불러서 "부모가 직접" 값을 출력하게 만들 수 있다.

- 모든 과정이 올바르게 실행되어, 원하는 값이 저장/출력된다.

에러 없음!

원리나 단계별 추가 설명이 필요하면, 각 단계의 변수/메서드 흐름을 예시로 더 분해해볼 수 있어요.

혹시 다른 조건(다른 생성자 사용, 변수 직접 접근 등) 상황을 알고 싶으면 추가로 질문 주세요!

✻

1. <https://non-stop.tistory.com/23>
2. <https://sy-kang.tistory.com/27>
3. <https://velog.io/@lyju777/JAVA-클래스의-상속과-생성자>
4. https://velog.io/@lily_1115/java-클래스-생성자-호출-순서
5. <https://mimah.tistory.com/entry/Java-상속-관계에서-생성자-호출-순서>
6. <https://jaynam.tistory.com/entry/JAVA-상속-관계-클래스-생성자-호출-순서-정리>
7. <https://chobo24.tistory.com/entry/JAVA자바-상속과-생성자super>
8. <https://blog.naver.com/namdda/222053120880>
9. <https://www.infllearn.com/community/questions/1396274/상속과-생성자-호출순서>
10. <https://august-jhy.tistory.com/208>